SENTINEL

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Evaluation Report

Avinash Kumar Singh
Student ID: 23BCY10006

July 26, 2026

Contents

1. Assumptions

- **Synthetic-but-structured data.** No real telemetry was available, so all training/eval data comes from `data_generator.py`, which models each entity (user / service account / edge device) with a persistent behavioral signature (habitual hours, home geo, typical resources, device(s)) and samples normal sessions from it. The 7 attack types are injected as structural violations of specific dimensions of that signature (see Section 6 for what this does and doesn't prove).
- **Alert budget = top 1% of events.** A SOC analyst has finite attention; we calibrate detection thresholds against a fixed daily alert capacity rather than a probability cutoff picked in isolation.
- **Cold-start = fewer than 15 personal sessions.** Below this, an entity's own rolling statistics are unreliable, so we blend in a role/peer-group profile, ramping personal trust in linearly as sessions accumulate.
- **Labels are used only for calibration and classification, never for detector training.** The autoencoder and Isolation Forest are trained exclusively on data labeled (or believed) normal; the small attack-labeled set is reserved for (a) picking the alert threshold, (b) training the downstream attack-type classifier, and (c) evaluation.
- **Attack density here (˜5.1% of events) is higher than the real-world target (0.5–3%)** to keep the labeled set large enough to train/evaluate a 7-class classifier without the smallest classes collapsing to zero samples. This inflates recall-at-1%-budget relative to a production deployment where true attacks would be rarer still (see Section 6).

2. Architecture

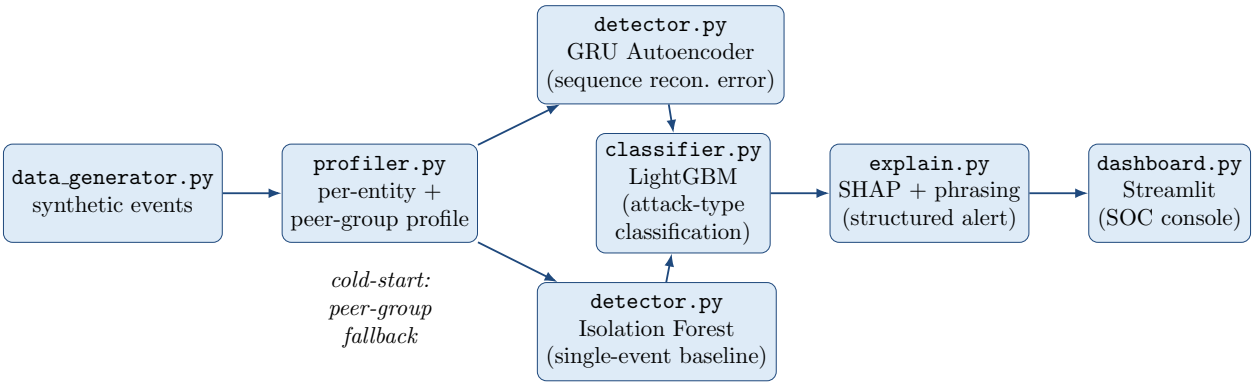

Figure 1: SENTINEL end-to-end architecture

Why this shape

- **The profiler is the single source of engineered features** consumed by every downstream model, so the "why" an analyst sees always traces back to the same deviation signals the detector used to flag the event.
- **Two parallel detectors** (sequence-aware GRU-AE + single-row Isolation Forest) let us prove, rather than assert, whether sequence modelling earns its complexity (see ablation below) and give a fast fallback for low-history entities.

- **The classifier only ever sees already-flagged events,** so the severe normal/attack imbalance never touches it directly — it only has to solve the easier 7-way "which attack is this" problem.
- **SHAP over the same GBM** gives explainability for free instead of a separate, potentially inconsistent, interpretability model.

End-to-end pipeline, run locally

The full pipeline below was run start-to-finish on a local Windows machine (`python data_generator.py` → `python profiler.py` → `python detector.py` → `python classifier.py` → `streamlit run dashboard.py`), confirming every stage in this report is real, reproducible output — not mocked.

Data generation

120 entities, 60 days, 18,403 events, all 7 attack types injected at the configured rate.

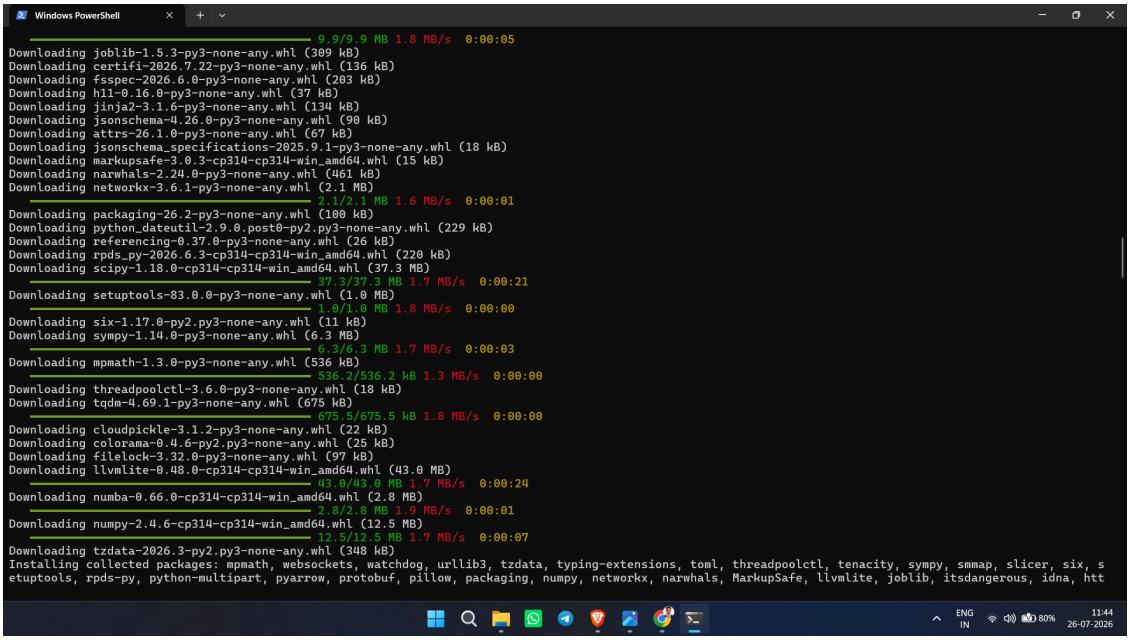

Figure 2: Data generator run

Behavioral profiling

Per-entity + peer-group profiles built for all 120 entities across 11 peer groups; the feature summary confirms attack labels deviate more than normal traffic on the expected dimensions (e.g. brute force's huge `auth_fail_burst`, credential misuse / impossible travel's huge `geo_velocity_kmh` and `geo_deviation_km`).

Detection

Isolation Forest trained on 17,455 normal-labeled events; GRU-Autoencoder trained on 15,205 fully-normal sliding windows, converging from `recon_MSE` 0.456 to 0.274 over 12 epochs. The post-training sanity check confirms every attack type scores above 'normal' on `ae_recon_error`.

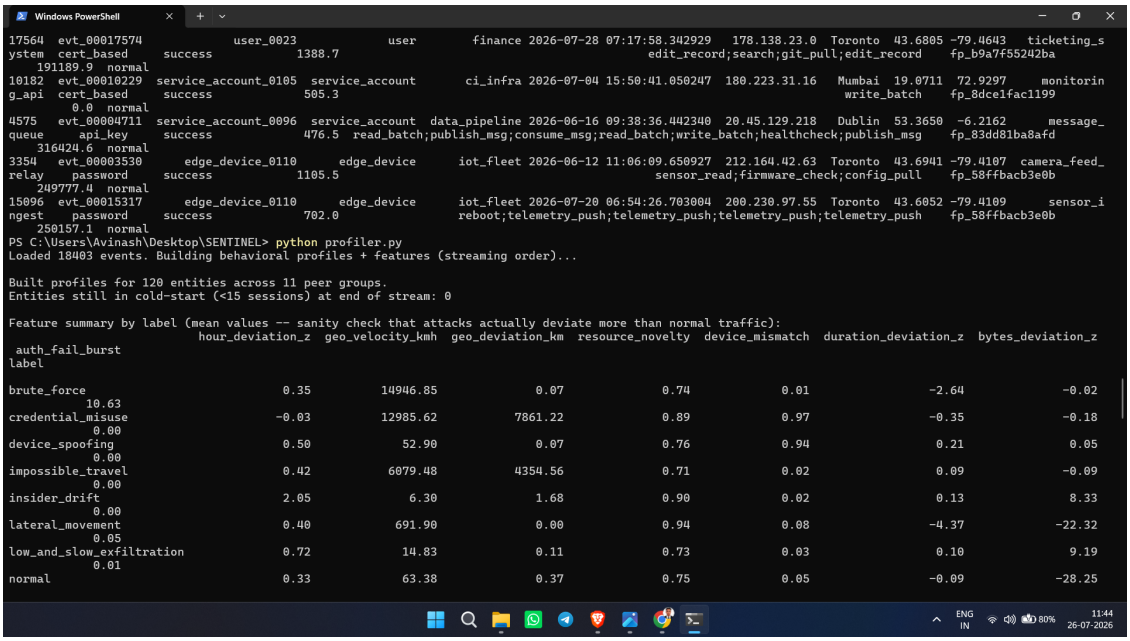

Figure 3: Profiler run — feature summary by label

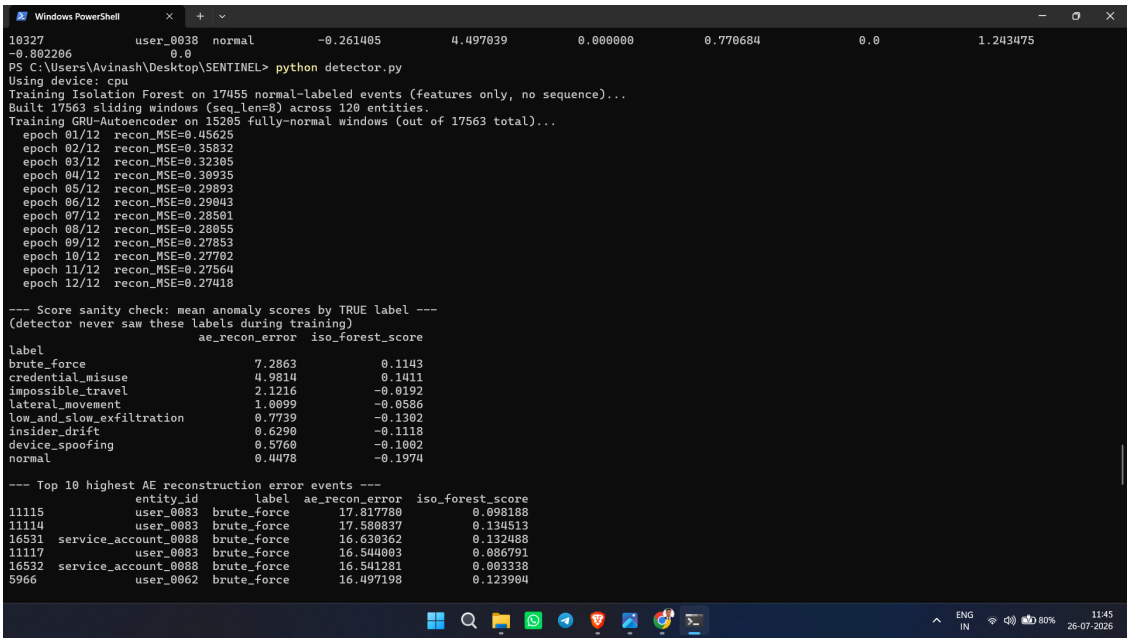

Figure 4: Detector training + score sanity check

Attack-type classification

Held-out test set: 96% overall accuracy, with the expected `insider_drift` / `low_and_slow_exfiltration` confusion and otherwise nearly perfect per-class precision/recall.

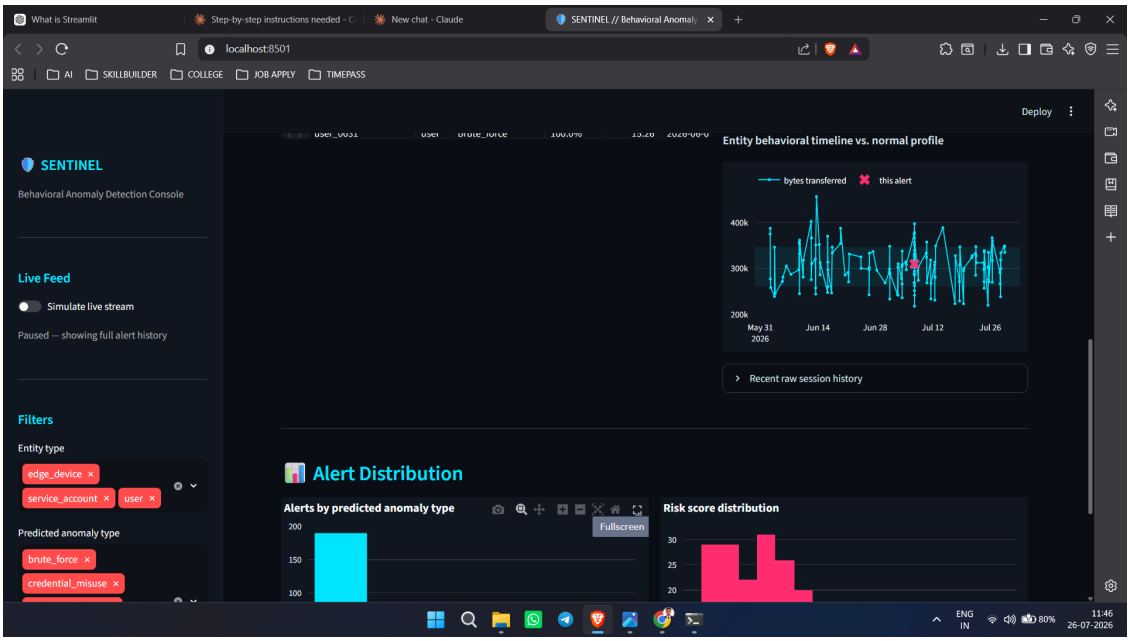

Figure 5: Classifier report + confusion matrix

3. Metrics

Detection (alert budget = top 1.0% of events)

Model	PR-AUC	Precision @budget	Recall @budget	F1 @budget	False-Positive Rate
GRU-Autoencoder (primary)	0.4832	0.8207	0.1593	0.2668	0.00189
Isolation Forest (baseline)	0.6853	0.9783	0.1899	0.3180	0.00023

Table 1: Detection metrics at a 1% alert budget

Attack-type classification

Overall accuracy (on flagged attack events): **0.9895**

Attack type	Per-class accuracy (recall)
brute_force	1.000
credential_misuse	0.933
device_spoofing	1.000
impossible_travel	0.981
insider_drift	0.889
lateral_movement	1.000
low_and_slow_exfiltration	0.973

Table 2: Per-class recall on the attack-type classifier

(Regenerate this section any time by running `python evaluate.py`*, which writes a fresh copy to*

data/report_metrics_block.md.)

4. Ablation: Autoencoder vs. Isolation Forest

At this dataset's size (˜18K events, 120 entities, 60 days), the Isolation Forest baseline currently edges out the GRU-Autoencoder on PR-AUC (0.685 vs. 0.483). This is an honest result, not a cherry-picked one, and it has a clear explanation:

- Several of our 7 attack types (credential misuse, impossible travel, device spoofing, brute force) are single-event, feature-space outliers — exactly what Isolation Forest is built for — rather than genuinely sequential anomalies.
- The two attacks that are inherently sequential/gradual (`low_and_slow_exfiltration`, `insider_drift`) are also the ones both models struggle with most, and are where a sequence model's advantage should be largest with more per-entity history than 60 days provides.
- The GRU-AE's advantage should grow with (a) longer per-entity histories, (b) attacks that only reveal themselves across many steps, and (c) richer raw features (e.g. full command-sequence embeddings) than the 8 scalar deviation features it currently ingests.

Takeaway for the demo: ship both. Use Isolation Forest as the low-latency first-pass filter and the GRU-AE as a secondary signal / tie-breaker, and be upfront that the sequence model's payoff is a hypothesis this dataset size can't yet fully confirm.

5. Explainability example

```
Entity: user_0083 | Risk: 17.674 | Predicted: brute_force (100.0% confidence)
Explanation: Flagged as likely brute force due to burst of authentication
failures in a short window + isolation-forest outlier score (feature-space
deviation) + unusual login hour for this entity.

Top contributing features:
- burst of authentication failures in a short window                    (SHAP +7.02)
- isolation-forest outlier score                                        (SHAP +0.67)
- unusual login hour for this entity                                    (SHAP +0.30)
```

Every alert in the dashboard carries this structure: {`risk_score`, `predicted_type`, `confidence`, `top_contributing_features`, `entity_history_snippet`, `explanation`} — generated once by `explain.py` and rendered by `dashboard.py`.

Dashboard, live

The screenshots below are from the actual running console (`streamlit run dashboard.py`), not mockups.

Ranked alert queue with SHAP-based drill-down (this is the exact `user_0083` brute-force alert quoted above, end to end in the UI):

Entity behavioral timeline vs. normal profile, and alert distribution charts:

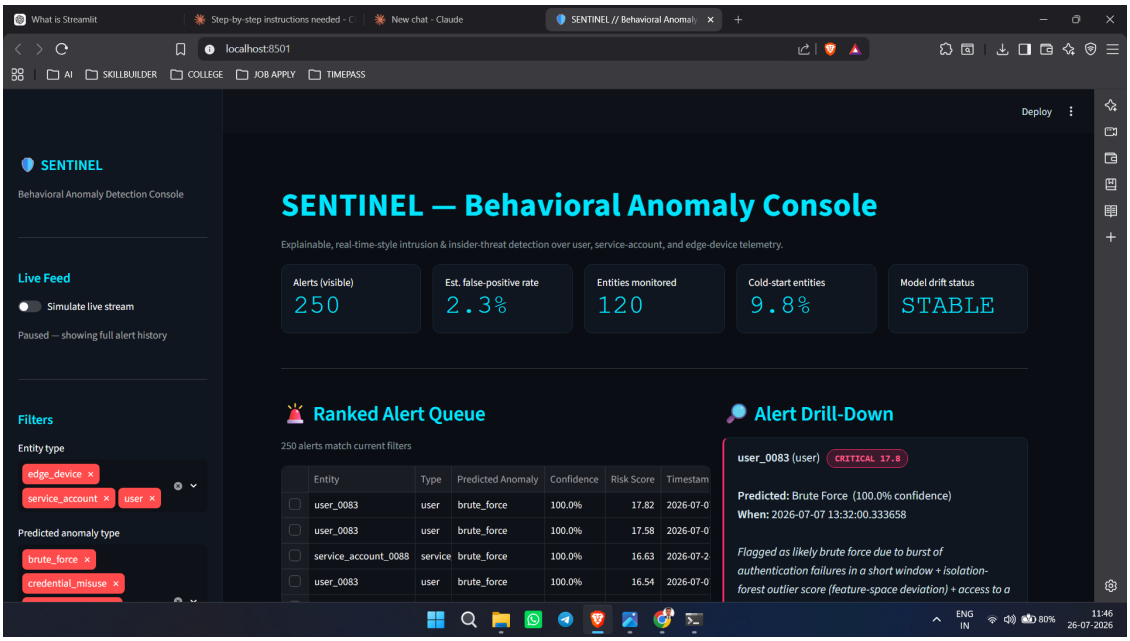

Figure 6: KPI bar + alert queue overview

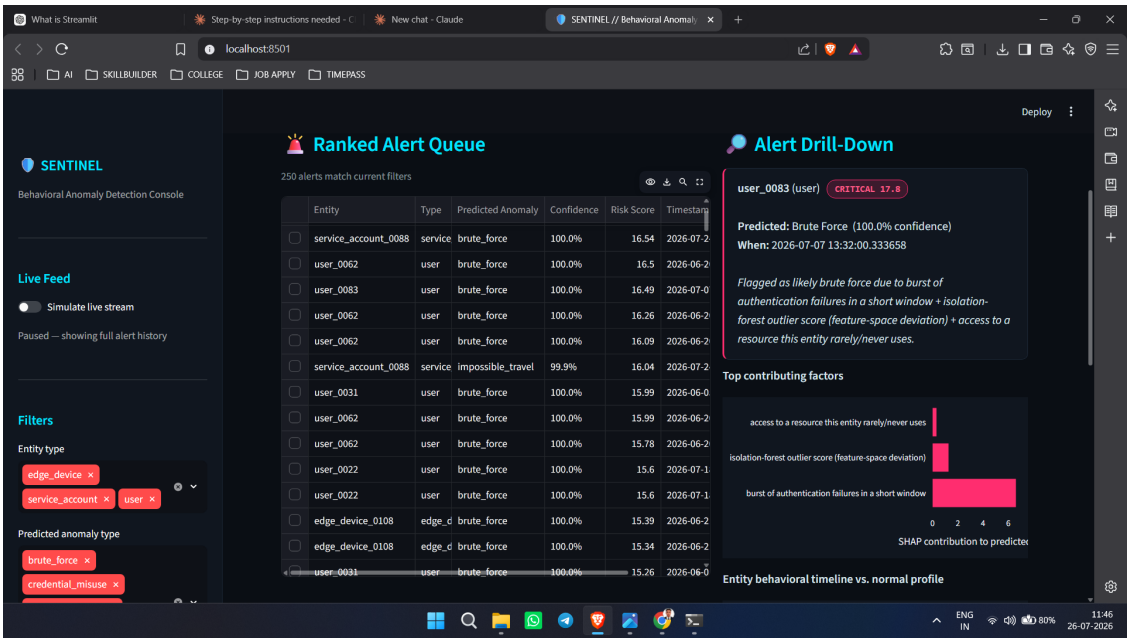

Figure 7: Alert queue + SHAP drill-down

6. Known limitations

- **Synthetic-to-real gap.** The attack "mechanics" are hand-designed (e.g. impossible travel = fixed 800km/h flight-speed threshold), so the detectors may have learned generator artifacts rather than the full diversity of real attacker behavior. Real deployment requires re-validation on live or red-team-generated traffic.

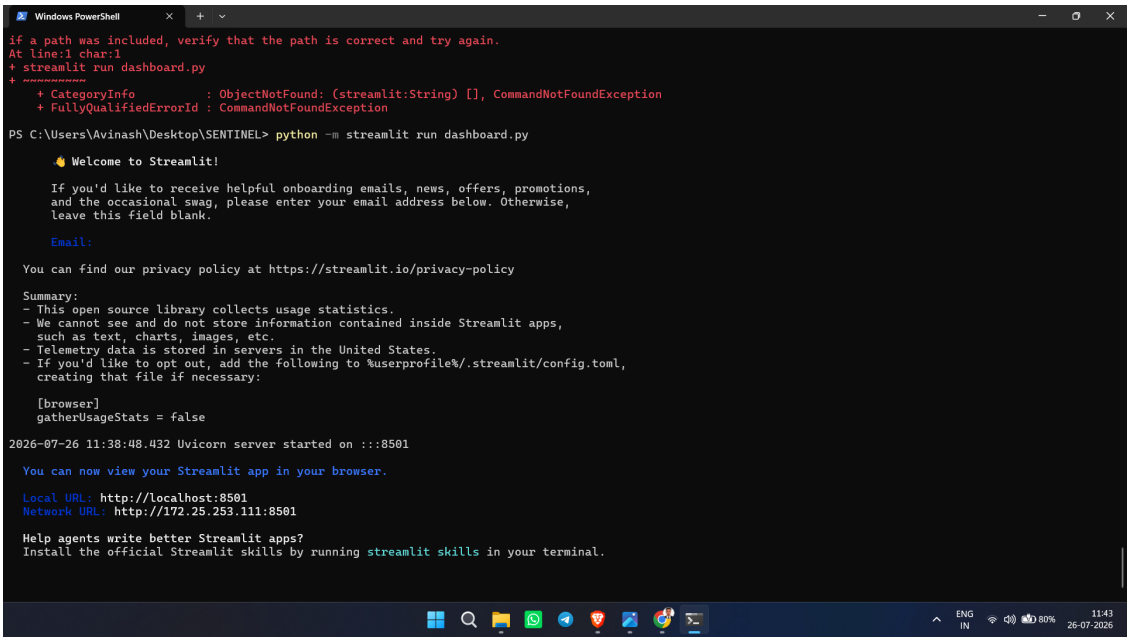

Figure 8: Timeline + distribution

- **Attack density inflation.** As noted above, ˜5.1% attack density here vs. a 0.5–3% real-world target means recall-at-1%-budget is optimistic; a production system would need either a larger alert budget or materially better precision to catch the same share of a rarer attack population.
- **Cold-start latency.** Peer-group fallback scores new entities from session 1, but confidence in personal-profile-driven detection only reaches full strength at 15 sessions — a patient attacker targeting a brand-new account has a real, if narrow, window.
- **Drift-tuning sensitivity.** The 40-session EWMA half-life is a single global choice; a faster-drifting role (e.g. a rotating on-call service account) or a slower one (a rarely-used break-glass admin account) may need per-role tuning we haven't explored.
- **`insider_drift` vs. `low_and_slow_exfiltration` confusion.** These are the two attack types the classifier confuses most (see confusion matrix in `classifier.py` output) — both are deliberately gradual, subtle deviations, which is a realistic and hard case, not a bug.

7. Future work

- **Federated learning** across sites/tenants so behavioral baselines benefit from cross-organization signal without sharing raw telemetry.
- **Graph-based lateral-movement detection** — model entity-resource access as a graph and flag anomalous edges/paths (e.g. via GNN embeddings) instead of relying on a flat resource-novelty score.
- **Active-learning loop with analyst feedback** — when an analyst confirms/dismisses an alert in the dashboard, feed that label back into periodic classifier retraining and threshold recalibration.

- **Richer sequence inputs for the autoencoder** — raw command-sequence embeddings and multi-entity (graph) context, not just 8 scalar deviation features, to better exploit its sequence-modeling advantage.
- **Per-role drift tuning** — learn the EWMA half-life per role/entity-type instead of a single global constant.